FastAPI Learn

# FastAPI CLI

**FastAPI CLI** is a command line program that you can use to serve your FastAPI app, manage your FastAPI project, and more.

When you install FastAPI (e.g. with `pip install "fastapi[standard]"`), it comes with a command line program you can run in the terminal.

To run your FastAPI app for development, you can use the `fastapi dev` command:

```bash
$ fastapi dev

  FastAPI   Starting development server 🚀

            Searching for package file structure from directories
with
            __init__.py files
            Importing from /home/user/code/awesomeapp

   module   🐍 main.py

     code   Importing the FastAPI app object from the module with
the
            following code:

            from main import app

      app   Using import string: main:app

   server   Server started at http://127.0.0.1:8000
   server   Documentation at http://127.0.0.1:8000/docs

      tip   Running in development mode, for production use:
            fastapi run

            Logs:

     INFO   Will watch for changes in these directories:
            ['/home/user/code/awesomeapp']
     INFO   Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C
to
            quit)
     INFO   Started reloader process [383138] using WatchFiles
     INFO   Started server process [383153]
     INFO   Waiting for application startup.
     INFO   Application startup complete.

                                                        restart ↻
```

> **Tip**
>
> For production you would use `fastapi run` instead of `fastapi dev`. 🚀

Internally, **FastAPI CLI** uses Uvicorn, a high-performance, production-ready, ASGI server. 😎

The `fastapi` CLI will try to detect automatically the FastAPI app to run, assuming it's an object called `app` in a file `main.py` (or a couple other variants).

But you can configure explicitly the app to use.

## Configure the app `entrypoint` in `pyproject.toml`

You can configure where your app is located in a `pyproject.toml` file like:

```
[tool.fastapi]
entrypoint = "main:app"
```

That `entrypoint` will tell the `fastapi` command that it should import the app like:

```
from main import app
```

If your code was structured like:

```
.
├── backend
│   ├── main.py
│   ├── __init__.py
```

Then you would set the `entrypoint` as:

```
[tool.fastapi]
entrypoint = "backend.main:app"
```

which would be equivalent to:

```
from backend.main import app
```

### `fastapi dev` with path

You can also pass the file path to the `fastapi dev` command, and it will guess the FastAPI app object to use:

```
$ fastapi dev main.py
```

But you would have to remember to pass the correct path every time you call the `fastapi` command.

Additionally, other tools might not be able to find it, for example the VS Code Extension or FastAPI Cloud, so it is recommended to use the `entrypoint` in `pyproject.toml`.

## `fastapi dev`

Running `fastapi dev` initiates development mode.

By default, **auto-reload** is enabled, automatically reloading the server when you make changes to your code. This is resource-intensive and could be less stable than when it's disabled. You should only use it for development. It also

listens on the IP address `127.0.0.1`, which is the IP for your machine to communicate with itself alone (`localhost`).

## `fastapi run`

Executing `fastapi run` starts FastAPI in production mode.

By default, **auto-reload** is disabled. It also listens on the IP address `0.0.0.0`, which means all the available IP addresses, this way it will be publicly accessible to anyone that can communicate with the machine. This is how you would normally run it in production, for example, in a container.

In most cases you would (and should) have a "termination proxy" handling HTTPS for you on top, this will depend on how you deploy your application, your provider might do this for you, or you might need to set it up yourself.

> **Tip**
>
> You can learn more about it in the deployment documentation.